Git Workflows

Branches, commits, merging, rebasing, and surviving team development

> Reference: git-scm.com/cheat-sheet

Agenda

- Creating and switching branches
- Staging, committing, and pushing
- Keeping your branch up to date with `main`
- Rebase vs Merge – when to use each
- Fixing divergent branches
- Common team issues

Creating a Branch

Create and switch immediately

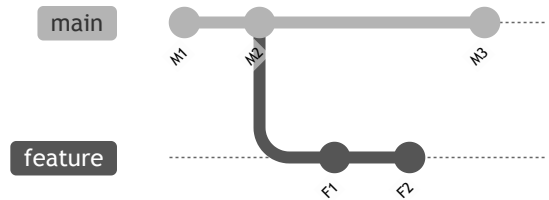
```
git switch -c <branch-name>
# older syntax
git checkout -b <branch-name>
```

Switch to existing branch

```
git switch <branch-name>
```

List / delete branches

```
git branch
git branch --sort=-committerdate

git branch -d <name>   # safe delete
git branch -D <name>   # force delete
```



Stage → Commit → Push

```
# 1. Stage changes
git add <file>           # specific file
git add .                # everything
git add -p               # pick hunks interactively

# 2. Check what's staged
git status

# 3. Commit
git commit -m 'your message'
git commit -am 'message'   # stage all tracked + commit

# 4. Push
git push -u origin <branch-name>   # first push (sets upstream)
git push                           # subsequent pushes
```

Keeping Your Branch Up to Date

Update your branch from `main` **regularly** to avoid large conflicts - two ways to do this

Updating via Merge

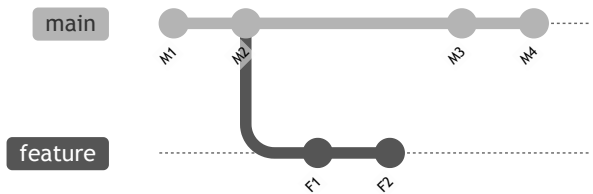
```
# Fetch latest from remote
git fetch origin main

# Switch to your feature branch
git switch feature

# Bring main's changes in
git merge main

# Or in one step
git pull origin main
```

What merge does:

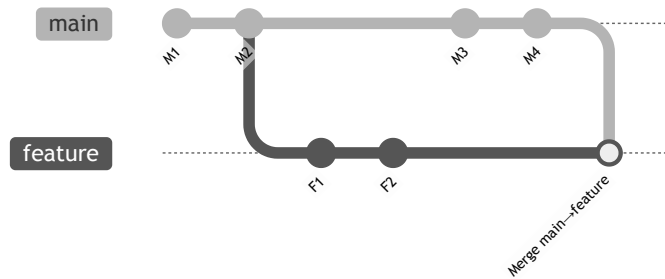
- Creates a **merge commit** joining both histories
- Preserves full branch history
- Safe on shared branches ✅

Merge: Before → After

Before – `feature` and `main` have both diverged



After `git merge main` (on feature branch)



Updating via Rebase

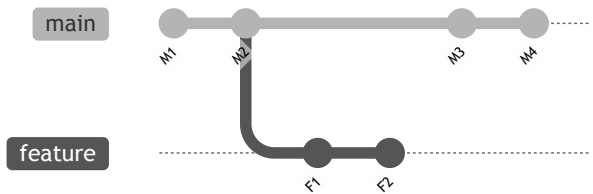
```
# Move your feature branch on top of the latest main
git switch feature
git rebase main

# Fetch latest changes, then rebase in one step
git pull --rebase

# Undo a failed rebase
git reflog feature           # find the commit before rebase
git reset --hard <commit>    # go back to it
```

What rebase does:

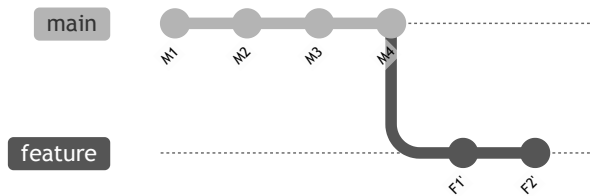
- Re-applies your commits on top of the tip of `main`
- Results in a **linear history** – no merge commits
- Rewrites commit SHAs ⚠️ treat as new commits
- Safe only on **local / unshared** branches

Rebase: Before → After

Before – `feature` branched from an older `main`



After `git rebase main` – F1/F2 replayed on top



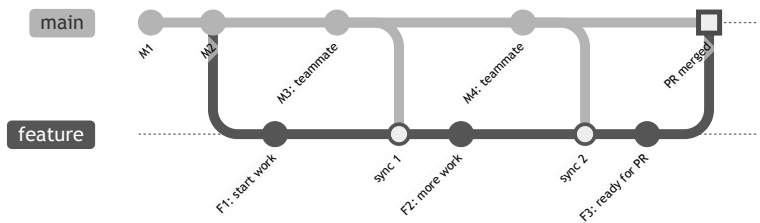
Rebase vs Merge

Two ways to integrate changes – with very different histories

Branch Lifecycle: Merge-only vs Rebase + Merge

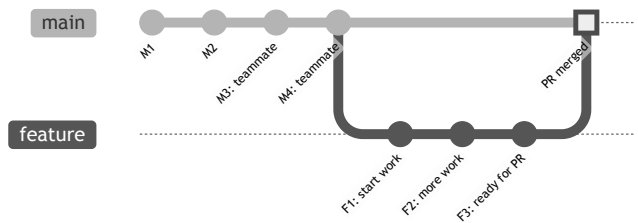
From branch creation → daily sync → PR approved → merged

⚠️ **Merge only** – extra sync commits pile up



Merging main into your branch isn't wrong, but is generally better to rebase if you are still the only one on the branch Some project mandate a clean branch history for PRs

✅ **Rebase during dev, merge at end** – clean history



Right diagram represents the state **after rebasing** onto the latest `main` before the PR – feature commits appear on top of M4.

Rebase vs Merge – Comparison

	Merge	Rebase
History	Full branching history	Linear, cleaner
Extra commits	Adds a merge commit	No extra commits
Rewrites history	❌ No	✅ Yes
Safe on shared branches	✅ Yes	⚠️ Avoid
Use for	Integrating features, PRs	Updating local branch

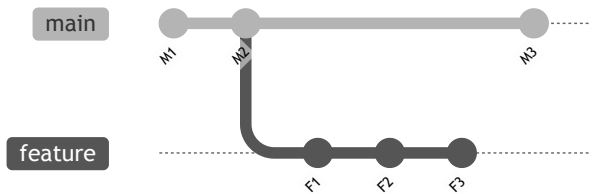
Team rules of thumb:

- ✅ **Rebase** your feature branch onto `main` before opening a PR
- ✅ **Merge** changes from `main` into your branch after the PR is up or the branch has been shared
- ❌ **Never** rebase a branch others are working from

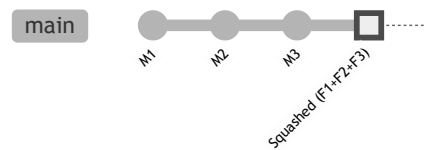
Squash Merge: Before → After

All feature commits collapsed into **one** commit on `main`

Before



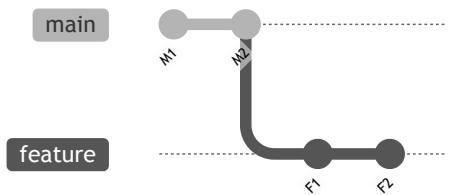
After `git merge --squash feature`



Fast-Forward Merge: Before → After

When `main` hasn't moved – Git just advances the pointer

Before – `main` hasn't changed since branch



After `git merge feature` – pointer moves forward

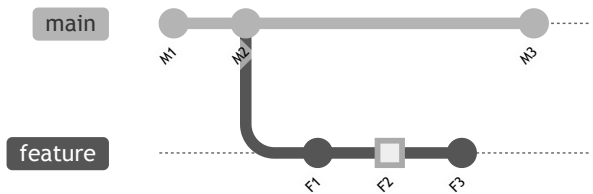
Fixing Divergent Branches

When both local and remote have commits the other doesn't have

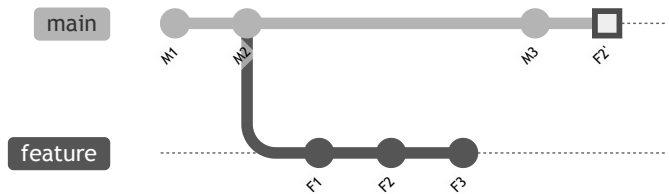
Cherry-Pick: Before → After

Copy a **single commit** from another branch

Before – commit `F2` is on `feature` only



After `git cherry-pick F2`



Resolving Common Issues

Merge conflicts – edit the conflict markers, then:

```
git add <resolved-file>
git commit           # after merge
git rebase --continue  # after rebase

git merge --abort    # give up on merge
git rebase --abort   # give up on rebase
```

Committed to the wrong branch

```
git reset HEAD^      # undo last commit, keep changes staged
git switch <correct-branch>
git commit -m 'message'
```

Safe force push (won't overwrite others' work)

```
git push --force-with-lease
```

More Common Fixes

Stash work in progress

```
git stash          # save changes without committing
git stash pop      # restore them later
```

Amend last commit (message or missed file)

```
git add <forgotten-file>
git commit --amend
```

Squash last 5 commits into one

```
git rebase -i HEAD~6
# Change "pick" → "fixup" for commits to squash
```

Ways to Refer to a Commit

Reference	Example	Meaning
Branch	`main`	Tip of the branch
Tag	`v1.0`	Named release
Commit ID	`3e887ab`	Exact commit
Remote branch	`origin/main`	Remote tip
Current commit	`HEAD`	Where you are now
N commits ago	`HEAD~3`	3 commits back

```
git diff HEAD~3        # diff against 3 commits ago
git checkout v1.0      # go to a tagged release
git reset origin/main  # reset to what's on remote
```

Summary

- `git switch -c` – create and move to a new branch
- `git add` → `git commit` → `git push` – the daily loop
- **Merge** `main` into your branch often to stay up to date
- **Rebase** before PRs for a clean linear history; **merge** to integrate
- `--force-with-lease` instead of `--force` to stay safe
- `git stash`, `git reset HEAD^`, `git reflog` – your escape hatches

> git-scm.com/cheat-sheet